The type you pass in to the `Ilogger<T>` to generate the category name is constrained to only those types that your application recognizes. That doesn't fit the *label a log with an arbitrary string* definition. There is another way to instantiate loggers that allows you to provide any string you like as the category name. You can use `IloggerFactory` to create instances of loggers instead. Using this approach, you inject `IloggerFactory` into the consuming class and then use its `CreateLogger` method to instantiate an instance of `Ilogger` as needed. The `CreateLogger` method takes a string to represent the logger category. The following listing shows the relevant amendments to the `CityModel` in bold. The constructor of the `CityModel` class is amended to accommodate an `Ilogger-Factory`, which is assigned to a field. The factory is used within the `OnGetAsync` method to instantiate an instance of `Ilogger` for use within that method only, with a suitable category name.

Listing 12.11 Using an `IloggerFactory` to customize the category name

```
private readonly IcityService _cityService;
private readonly IpropertyService _propertyService;
private readonly IloggerFactory _loggerFactory;                    ❶

public CityModel(IcityService cityService,
    IpropertyService propertyService,
    IloggerFactory loggerFactory)                                  ❶
{
    _cityService = cityService;
    _propertyService = propertyService;
    _loggerFactory = loggerFactory;                                ❶
}

[BindProperty(SupportsGet = true)]
public string Name { get; set; }
public City City { get; set; }

public async Task<IactionResult> OnGetAsync()
{
    var logger = _loggerFactory.CreateLogger("City page OnGetAsync");  ❷
    City = await _cityService.GetByNameAsync(Name);
```